COMP3242/6242: Deep Learning

Week 10 — Deep Reinforcement Learning

Stephen Gould
stephen.gould@anu.edu.au

Australian National University

Semester 1, 2026

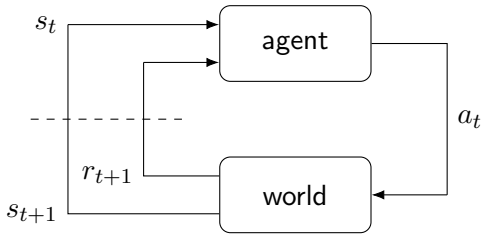
Overview

- Sequential decision making
- Policies and value functions
  - Markov decision process
- Reinforcement learning
  - Exploration versus exploitation
- Deep reinforcement learning

Reading and Resources.

- *Russell and Norvig*, Artificial Intelligence: A Modern Approach, 1995
- *Sutton and Barto*, Reinforcement Learning: An Introduction, 2014
- *Mnih et al.*, Playing Atari with Deep Reinforcement Learning, 2013
- *Mnih et al.*, Human-level Control through Deep Reinforcement Learning, 2015

Sequential Decision Making



- Agent observes the current state of the world s_t
- Agent decides on an action to take a_t
- As a result the state of the world changes s_{t+1}
- Agent is rewarded (or punished) r_{t+1}

Policies, Rewards, and Value Functions

Let $\mathcal{S}$ be the set of (all possible) states and $\mathcal{A}$ be the set of (all possible) actions

- A **policy** is a function that maps states to actions

$$\pi : \mathcal{S} \to \mathcal{A}$$

Policies, Rewards, and Value Functions

Let $\mathcal{S}$ be the set of (all possible) states and $\mathcal{A}$ be the set of (all possible) actions

- A **policy** is a function that maps states to actions

$$\pi : \mathcal{S} \to \mathcal{A}$$

- A **reward function** maps each state (or state-action pair) to a real number

$$R : \mathcal{S} \to \mathbb{R}$$

Policies, Rewards, and Value Functions

Let $\mathcal{S}$ be the set of (all possible) states and $\mathcal{A}$ be the set of (all possible) actions

- A **policy** is a function that maps states to actions

$$\pi : \mathcal{S} \to \mathcal{A}$$

- A **reward function** maps each state (or state-action pair) to a real number

$$R : \mathcal{S} \to \mathbb{R}$$

- A **value function** is the total expected reward by following a policy starting from a particular state

$$V_\pi : \mathcal{S} \to \mathbb{R}$$

Policies, Rewards, and Value Functions

Let $\mathcal{S}$ be the set of (all possible) states and $\mathcal{A}$ be the set of (all possible) actions

- A **policy** is a function that maps states to actions

$$\pi : \mathcal{S} \to \mathcal{A}$$

- A **reward function** maps each state (or state-action pair) to a real number

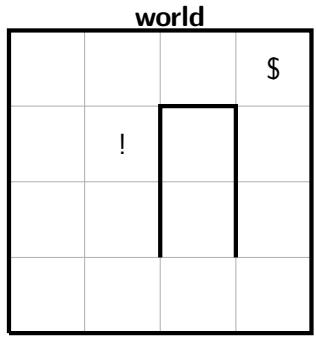
$$R : \mathcal{S} \to \mathbb{R}$$

- A **value function** is the total expected reward by following a policy starting from a particular state

$$V_\pi : \mathcal{S} \to \mathbb{R}$$

- a **Markov decision process (MDP)** is a tuple $(\mathcal{S}, \mathcal{A}, P(s_{t+1} \mid s_t, a_t), \gamma, R)$
  - γ is a discount factor (which we'll discuss later)

Example: Grid World



- state is the agent's (x, y) location
- actions are $\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow, \circ\}$

Example: Grid World

reward function, $R(s)$

-1	-1	-1	10
-1	-10	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

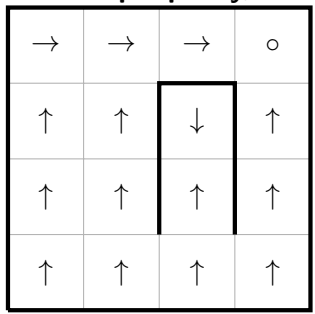
- state is the agent's (x, y) location
- actions are $\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow, \circ\}$

Example: Grid World

reward function, $R(s)$

-1	-1	-1	10
-1	-10	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

example policy, π



- state is the agent's (x, y) location
- actions are $\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow, \circ\}$

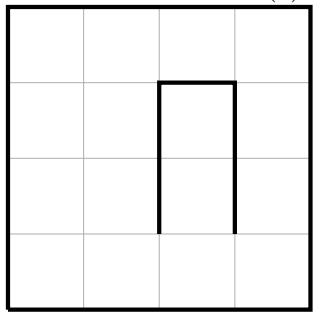
- go north if can, else east, else south, else west, stop at (4,4)

Example: Grid World

reward function, $R(s)$

-1	-1	-1	10
-1	-10	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

value function, $V_\pi(s)$



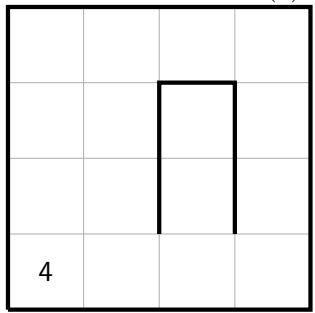
- π: go north if can, else east, else south, else west, stop at (4,4)

Example: Grid World

reward function, $R(s)$

-1	-1	-1	10
-1	-10	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

value function, $V_\pi(s)$



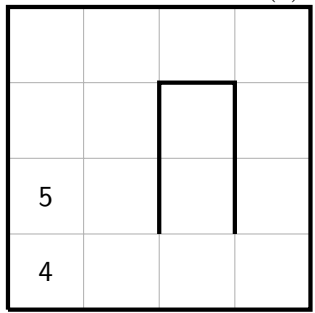
- π: go north if can, else east, else south, else west, stop at (4,4)

Example: Grid World

reward function, $R(s)$

-1	-1	-1	10
-1	-10	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

value function, $V_\pi(s)$



- π: go north if can, else east, else south, else west, stop at (4,4)

Example: Grid World

reward function, $R(s)$

-1	-1	-1	10
-1	-10	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

value function, $V_\pi(s)$

7	8	9	10
6			
5			
4			

- π: go north if can, else east, else south, else west, stop at (4,4)

Example: Grid World

reward function, $R(s)$

-1	-1	-1	10
-1	-10	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

value function, $V_\pi(s)$

7	8	9	10
6	-2	$-\infty$	9
5	-3	$-\infty$	8
4	-4	$-\infty$	7

- π: go north if can, else east, else south, else west, stop at (4,4)
- different policies will have different value functions

Supervised Learning vs Reinforcement Learning

Supervised Learning

- learns a function from input space to output space

$$f_\theta : \mathcal{X} \to \mathcal{Y}$$

- predictions can be evaluated immediately
- f_θ is chosen to minimize loss
- training data is usually i.i.d.

Supervised Learning vs Reinforcement Learning

Supervised Learning

- learns a function from input space to output space

$$f_\theta : \mathcal{X} \to \mathcal{Y}$$

- predictions can be evaluated immediately
- f_θ is chosen to minimize loss
- training data is usually i.i.d.

Reinforcement Learning

- learns a function from states to actions (behaviour)

$$\pi_\theta : \mathcal{S} \to \mathcal{A}$$

- predicted action cannot be evaluated directly
  - rather they receive an interim positive/negative reward
- π_θ is chosen to maximize total (discounted) **expected reward**
- training data is not i.i.d. — actions a affect future states

Why Maximize Expected Reward?

- the world is stochastic, i.e., non-deterministic dynamics

$$s_{t+1} \sim P(s' \mid s_t, a_t)$$

- policies may be probabilistic

$$a_t \sim \pi_\theta(a \mid s_t)$$

- in general, reward can depend on the transition

$$r_t = R(s_t, a_t, s_{t+1})$$

  but we'll keep things simple and only consider $R(s)$

Return and Discount Factors

- **return** (or reward-to-go) is the discounted sum of rewards from time step t

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

- discount factor avoids infinite returns
  - if $\gamma = 0$ then we only care about the immediate reward
  - if $\gamma = 1$ then the future reward is as important as the immediate reward
  - humans often behave as if $0 < \gamma < 1$
  - if episodes are finite then we can take $\gamma = 1$

Return and Discount Factors

- **return** (or reward-to-go) is the discounted sum of rewards from time step t

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

- discount factor avoids infinite returns
  - if $\gamma = 0$ then we only care about the immediate reward
  - if $\gamma = 1$ then the future reward is as important as the immediate reward
  - humans often behave as if $0 < \gamma < 1$
  - if episodes are finite then we can take $\gamma = 1$
- the value function is then

$$\begin{aligned} V_\pi(s) &= E\left[G_t \mid s_t = s\right] \\ &= R(s) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s) V(s') \end{aligned}$$

Q-Function

- value function is the expected return starting in state s and following policy π thereafter,

$$V_\pi(s) = E\left[G_t \mid s_t = s\right]$$

- q-function is the expected return starting in state s, taking action a, and following policy π thereafter,

$$Q_\pi(s, a) = E\left[G_t \mid s_t = s, a_t = a\right]$$

- we can obtain the value function from the q-function

$$V_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) Q_\pi(s, a)$$

- both V_π and Q_π can be estimated from experience

Trajectories and Roll-outs

- expectations are taken with respect to **trajectories**,

$$\tau = (s_1, a_1, s_2, a_2, \ldots, s_T, a_T)$$

- when a policy π_θ is used to generate a trajectory we call it a **roll-out** or **episode**
- so the value function is

$$V_\pi(s) = E_{\tau \sim P_\theta(\tau | s_1 = s)} \left[\sum_{t=1}^{T} \gamma^{t-1} R(s_t) \right]$$

where $P_\theta(\tau) = P(s_1) \prod_{t=1}^{T} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t)$

Optimal Policy

- an **optimal policy** π_* is one that leads to better rewards than any other policy π,

$$V_*(s) = \max_\pi V_\pi(s) = \max_\pi E\left[G_t \mid s_t = s\right]$$
$$Q_*(s,a) = \max_\pi Q_\pi(s,a) = \max_\pi E\left[G_t \mid s_t = s, a_t = a\right]$$

- given Q_* we can obtain the optimal policy as

$$\pi_*(s) = \text{argmax}_a\, Q_*(s,a)$$

- Bellman equation — the optimal value is the current reward plus the optimal value starting at the next state

$$Q_*(s,a) = E_{s' \sim P(s'|s,a)}\left[R(s) + \gamma \max_{a'} Q_*(s',a') \mid s, a\right]$$

Example: Grid World Optimal Policy

reward function, $R(s)$

-1	-1	-1	10
-1	-10	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

optimal policy, π_*

$\rightarrow$	$\rightarrow$	$\rightarrow$	$\circ$
$\uparrow$	$\uparrow$	$\downarrow$	$\uparrow$
$\uparrow$	$\downarrow$	$\downarrow$	$\uparrow$
$\uparrow$	$\rightarrow$	$\rightarrow$	$\uparrow$

- must stop at and only at (4,4)

Value Iteration

- value iteration is a dynamic programming method for finding V_* or Q_* by iterating the Bellman equations
  - synchronous or asynchronous (can overwrite Q each update)

$Q_0(s,a) \leftarrow 0$ for all s and all a
for $i = 1$ **to** ∞ **do**
  for $s \in \mathcal{S}$ **do**
    for $a \in \mathcal{A}$ **do**
      $Q_i(s,a) = \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left(R(s) + \gamma \max_{a' \in \mathcal{A}} Q_{i-1}(s',a')\right)$
    end
  end
end

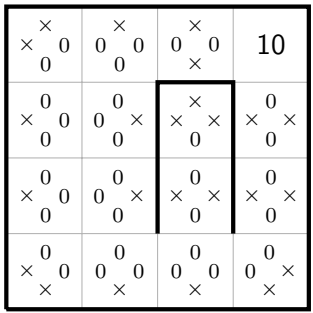
- Q_i converges to Q_* as $i \to \infty$

Example: Grids World Value Iteration for Q_* with $\gamma = 1$

reward function, $R(s)$

-1	-1	-1	10
-1	-10	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

q-function, Q_0



- must stop at and only at (4,4)
  - otherwise can get infinite award

Example: Grids World Value Iteration for Q_* with $\gamma = 1$

reward function, $R(s)$

-1	-1	-1	10
-1	-10	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

q-function, Q_1

$\times$ $\times\ -1$ -1	$\times$ $-1\ -1$ -1	$\times$ $-1\ 9$ $\times$	10
-1 $\times\ -1$ -1	-10 $-10\ \times$ -10	$\times$ $\times\ \times$ -1	9 $\times\ \times$ -1
-1 $\times\ -1$ -1	-1 $-1\ \times$ -1	-1 $\times\ \times$ -1	-1 $\times\ \times$ -1
-1 $\times\ -1$ $\times$	-1 $-1\ -1$ $\times$	-1 $-1\ -1$ $\times$	-1 $-1\ \times$ $\times$

- must stop at and only at (4,4)
  - otherwise can get infinite award

- $Q_i(s,a) = R(s) + \max_{a'} Q_{i-1}(s', a')$

Example: Grids World Value Iteration for Q_* with $\gamma = 1$

reward function, $R(s)$

-1	-1	-1	10
-1	-10	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

q-function, Q_2

$\times$ / $\times$ -2 / -2	$\times$ / -2 8 / -11	$\times$ / -2 9 / $\times$	10
-2 / $\times$ -11 / -2	-11 / -11 $\times$ / -11	$\times$ / $\times$ $\times$ / -2	9 / $\times$ $\times$ / -2
-2 / $\times$ -2 / -2	-11 / -2 $\times$ / -2	-2 / $\times$ $\times$ / -2	8 / $\times$ $\times$ / -2
-2 / $\times$ -2 / $\times$	-2 / -2 -2 / $\times$	-2 / -2 -2 / $\times$	-2 / -2 $\times$ / $\times$

- must stop at and only at (4,4)
  - otherwise can get infinite award

- $Q_i(s, a) = R(s) + \max_{a'} Q_{i-1}(s', a')$

Example: Grids World Value Iteration for Q_* with $\gamma = 1$

reward function, $R(s)$

-1	-1	-1	10
-1	-10	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

q-function, Q_3

$\times$ $\times$ 7 -3	$\times$ -3 8 -12	$\times$ 7 9 $\times$	10
-3 $\times$ -12 -3	-2 -12 $\times$ -12	$\times$ $\times$ $\times$ -3	9 $\times$ $\times$ 7
-3 $\times$ -3 -3	-12 -3 $\times$ -3	-3 $\times$ $\times$ -3	8 $\times$ $\times$ -3
-3 $\times$ -3 $\times$	-3 -3 -3 $\times$	-3 -3 -3 $\times$	7 -3 $\times$ $\times$

- must stop at and only at (4,4)
  - otherwise can get infinite award

- $Q_i(s,a) = R(s) + \max_{a'} Q_{i-1}(s', a')$

Example: Grids World Value Iteration for Q_* with $\gamma = 1$

reward function, $R(s)$

-1	-1	-1	10
-1	-10	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

q-function, Q_4

$\times$ / $\times$ 7 / -4	$\times$ / 6 8 / -3	$\times$ / 7 9 / $\times$	10
6 / $\times$ -3 / -4	-2 / -13 $\times$ / -13	$\times$ / $\times$ $\times$ / -4	9 / $\times$ $\times$ / 7
-4 / $\times$ -4 / -4	-3 / -4 $\times$ / -4	-4 / $\times$ $\times$ / -4	8 / $\times$ $\times$ / 6
-4 / $\times$ -4 / $\times$	-4 / -4 -4 / $\times$	-4 / -4 6 / $\times$	7 / -4 $\times$ / $\times$

- must stop at and only at (4,4)
  - otherwise can get infinite award

- $Q_i(s, a) = R(s) + \max_{a'} Q_{i-1}(s', a')$

Example: Grids World Value Iteration for Q_* with $\gamma = 1$

reward function, $R(s)$

-1	-1	-1	10
-1	-10	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

q-function, Q_5

$\times$ / $\times$ 7 / 5	$\times$ / 6 8 / -3	$\times$ / 7 9 / $\times$	10
6 / $\times$ -3 / -5	-2 / -4 $\times$ / -13	$\times$ / $\times$ $\times$ / -5	9 / $\times$ $\times$ / 7
5 / $\times$ -4 / -5	-3 / -5 $\times$ / -5	-5 / $\times$ $\times$ / 5	8 / $\times$ $\times$ / 6
-5 / $\times$ -5 / $\times$	-4 / -5 5 / $\times$	-5 / -5 6 / $\times$	7 / 5 $\times$ / $\times$

- must stop at and only at (4,4)
  - otherwise can get infinite award

- $Q_i(s,a) = R(s) + \max_{a'} Q_{i-1}(s', a')$

Example: Grids World Value Iteration for Q_* with $\gamma = 1$

reward function, $R(s)$

-1	-1	-1	10
-1	-10	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

q-function, Q_6

$\times$ / $\times$ 7 / 5	$\times$ / 6 8 / -3	$\times$ / 7 9 / $\times$	10
6 / $\times$ -3 / 4	-2 / -4 $\times$ / -13	$\times$ / $\times$ $\times$ / 4	9 / $\times$ $\times$ / 7
5 / $\times$ -4 / -6	-3 / 4 $\times$ / 4	-6 / $\times$ $\times$ / 5	8 / $\times$ $\times$ / 6
-5 / $\times$ 4 / $\times$	-4 / -6 5 / $\times$	4 / 4 6 / $\times$	7 / 5 $\times$ / $\times$

- must stop at and only at (4,4)
  - otherwise can get infinite award

- $Q_i(s, a) = R(s) + \max_{a'} Q_{i-1}(s', a')$

Example: Grids World Value Iteration for Q_* with $\gamma = 1$

reward function, $R(s)$

-1	-1	-1	10
-1	-10	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

q-function, Q_7

$\times$ / $\times$ 7 / 5	$\times$ / 6 8 / -3	$\times$ / 7 9 / $\times$	10
6 / $\times$ -3 / 4	-2 / -4 $\times$ / -6	$\times$ / $\times$ $\times$ / 4	9 / $\times$ $\times$ / 7
5 / $\times$ 3 / 3	-3 / 4 $\times$ / 4	3 / $\times$ $\times$ / 5	8 / $\times$ $\times$ / 6
4 / $\times$ 4 / $\times$	3 / 3 5 / $\times$	4 / 4 6 / $\times$	7 / 5 $\times$ / $\times$

- must stop at and only at (4,4)
  - otherwise can get infinite award

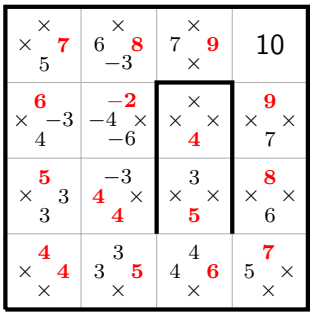
- $Q_i(s,a) = R(s) + \max_{a'} Q_{i-1}(s', a')$

Example: Grids World Value Iteration for Q_* with $\gamma = 1$

reward function, $R(s)$

-1	-1	-1	10
-1	-10	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

q-function, Q_*



- must stop at and only at (4,4)
  - otherwise can get infinite award

- $\pi_*(s) = \text{argmax}_a \, Q_*(s, a)$

Temporal-Difference (TD) Learning

- value iteration requires knowledge of $P(s' \mid s, a)$ and iteration over **all** states and actions

$$Q_i(s,a) = \sum_{s' \in \mathcal{S}} P(s' \mid s, a) \left(R(s) + \gamma \max_{a' \in \mathcal{A}} Q_{i-1}(s', a') \right)$$

- if we don't know how the world works so need to perform actions to find out
- TD learning uses sampled (s, a, s')-triplets to update value or Q-function

$$Q_i(s,a) = Q_{i-1}(s,a) + \alpha \left(\underbrace{R(s) + \gamma \max_{a' \in \mathcal{A}} Q_{i-1}(s', a')}_{\text{actual reward wrt } Q_{i-1}} - \overbrace{Q_{i-1}(s,a)}^{\text{predicted reward}} \right)$$

where α is a learning rate parameter

- stochastic (Monte Carlo) approximation of the Bellman equation

Exploration versus Exploitation

- as an agent learns it must decide between
  - taking the action that seems best right now (exploit), or
  - trying something new that might lead to better rewards (explore)

Exploration versus Exploitation

- as an agent learns it must decide between
  - taking the action that seems best right now (exploit), or
  - trying something new that might lead to better rewards (explore)

- exploitation, $a = \text{argmax}_{a \in \mathcal{A}} Q(s, a)$,
  - maximizes immediate reward (under the current model)
  - may miss better actions not yet tried

- exploration, $a \sim P(\mathcal{A})$,
  - lets the agent gather information about the world
  - may reduce short-term reward

Exploration versus Exploitation

- as an agent learns it must decide between
  - taking the action that seems best right now (exploit), or
  - trying something new that might lead to better rewards (explore)

- exploitation, $a = \text{argmax}_{a \in \mathcal{A}} Q(s, a)$,
  - maximizes immediate reward (under the current model)
  - may miss better actions not yet tried
- exploration, $a \sim P(\mathcal{A})$,
  - lets the agent gather information about the world
  - may reduce short-term reward

- if you always exploit you may get stuck with a sub-optimal policy
- if you always explore you don't capitalize on what you've learned
- this is a (mathematically) difficult trade-off
- one common solution is called the ϵ-greedy approach
  - with probability ϵ explore, otherwise exploit

Deep Learning RL Methods

- big idea: the state space is too large so learn mappings as neural networks rather than big lookup tables
- ultimate goal is to determine a policy that maximizes the expected sum of rewards

$$\text{maximize}_\theta \quad E_{\tau \sim P_\theta(\tau)} \left[\sum_{t=1}^{T} \gamma^{t-1} R(s_t, a_t) \right]$$

where $P_\theta(\tau) = P(s_1) \prod_{t=1}^{T} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t)$

- many methods have been developed
  - **imitation learning:** mimic a policy that achieves high reward
  - **policy gradient:** directly differentiate the above objective
  - **value-based:** estimate value of the optimal policy (e.g, deep Q-learning)
  - **actor-critic:** estimate value of the current policy and use it to improve
  - **model-based:** learn to model the world, and use it for planning

Imitation Learning (aka Learning from Demonstration)

- not really reinforcement learning (no exploration/exploitation) but can still be used to learn policies
- approximates $\pi_\theta : \mathcal{S} \to P(\mathcal{A})$ with a neural network
- collect examples of good policies (e.g., by recording an expert)
- treats problem as a classic supervised machine learning problem, e.g.,

$$\text{minimize (over } \theta) \quad -\sum_{(s,a)\in\mathcal{D}} \log \pi_\theta(a \mid s)$$

  where $\mathcal{D}$ is a dataset of (observed) state-action pairs

- imitation learning is conceptually simple but suffers from several limitations:
  - **distributional shift**: states encountered by agent may differ from those appearing in training dataset (i.e., not observed in expert demonstrations)
  - **causal confusion**: agent may correlate actions with irrelevant cues
  - **no rewards**: agent can't improve beyond the level of the expert demonstrator

Policy Gradient

- also approximates $\pi_\theta : \mathcal{S} \to P(\mathcal{A})$ with a neural network
- run policy to collect N trajectories; use trajectories to improve policy

$$\nabla_\theta L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \overbrace{\log \pi_\theta(a_{i,t} \mid s_{i,t})}^{\text{policy log-likelihood}} \left(\underbrace{\left(\sum_{\tau=t}^{T} \gamma^{\tau-t} r_{i,\tau} \right) - b}_{\substack{\text{reward to go, } G, \\ \text{relative to baseline}}} \right)$$

where the baseline b is used to reduce the variance of the gradient estimator

- a common choice for b is a guess for the value of state $s_{i,t}$ (see Actor-Critic later)

- very inefficient, unstable training (due to noisy gradients), get stuck easily

Deep Q-Learning

(Mnih et al., 2013, 2015)

- approximates $Q_\pi : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ with a neural network
- optimizes a loss reminiscent of TD learning using SGD,

$$L(\theta) = \sum_{(s,a,r,s') \in \mathcal{D}} \left(r + \gamma \max_{a'} Q(s', a'; \theta^{\text{prev}}) - Q(s, a; \theta) \right)^2$$

where θ^{prev} are the network parameters from a previous training step

- updated every C steps and held fixed between updates

Deep Q-Learning

(Mnih et al., 2013, 2015)

- approximates $Q_\pi : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ with a neural network
- optimizes a loss reminiscent of TD learning using SGD,

$$L(\theta) = \sum_{(s,a,r,s') \in \mathcal{D}} \left(r + \gamma \max_{a'} Q(s', a'; \theta^{\text{prev}}) - Q(s, a; \theta) \right)^2$$

where θ^{prev} are the network parameters from a previous training step

- updated every C steps and held fixed between updates

- initially demonstrated on Atari 2600 games
  - the state is a stack of the last four greyscale frames rescaled to 64-by-64 pixels
  - number of actions varies from 4 to 18 depending on the game
  - simple CNN architecture with 3 convolutional layers
  - trained for about 38 days of game experience (50 million frames)
- predecessor to AlphaGo, AlphaZero and AlphaFold (although these use a mix of methods and are not pure Q learning)

Deep Q-Learning — Experience Replay

(Mnih et al., 2013, 2015)

- sequential interactions are highly correlated leading to poor learning
  - actions are taken via a ϵ-greedy policy
- instead of directly learning from immediate interactions the DQN algorithm stores experiences in a **replay buffer**

$$\mathcal{D} = \{(s, a, r, s')\}$$

- randomly samples mini-batches for SGD from this buffer
- (s, a, r, s') tuples can be used multiple times $\rightarrow$ better sample efficiency
- together with the slowly moving target network (i.e., $Q(\cdot, \cdot; \theta^{\text{prev}})$) in the loss, experience replay helped to stabilize training making deep Q-learning effective

Actor-Critic Models — Policy Gradient Baselines

- combines policy gradient and value-based methods (e.g., DQN)
  - faster, more stable, explicit policy, policy can be stochastic

Actor-Critic Models — Policy Gradient Baselines

- combines policy gradient and value-based methods (e.g., DQN)
  - faster, more stable, explicit policy, policy can be stochastic
- recall that the baseline b reduces variance in policy gradient estimates

$$\nabla_\theta L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \left(\left(\sum_{\tau=t}^{T} \gamma^{\tau-t} r_{i,\tau} \right) - b \right)$$

Actor-Critic Models — Policy Gradient Baselines

- combines policy gradient and value-based methods (e.g., DQN)
  - faster, more stable, explicit policy, policy can be stochastic
- recall that the baseline b reduces variance in policy gradient estimates

$$\nabla_\theta L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \left(\left(\sum_{\tau=t}^{T} \gamma^{\tau-t} r_{i,\tau} \right) - b \right)$$

- a good choice for b is the average reward for state s, so we can write

$$\nabla_\theta L(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \big(Q_\pi(s_{i,t}, a_{i,t}) - V_\pi(s_{i,t}) \big)$$

where we have also replaced the reward to go with Q_π, its expectation

Actor-Critic Models — Advantage

$$\nabla_\theta L(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \big(Q_\pi(s_{i,t}, a_{i,t}) - V_\pi(s_{i,t}) \big)$$

- define **advantage** to measure the relative value of choosing action a in state s over other actions,

$$\begin{aligned} A_\pi(s,a) &= Q_\pi(s,a) - V_\pi(s) \\ &\approx R(s,a) + \gamma V_\pi(s') - V_\pi(s) \end{aligned}$$

  since $Q_\pi(s,a) = R(s,a) + \gamma E_{s' \sim P(\cdot|s,a)} \left[V_\pi(s') \right] \approx R(s,a) + \gamma V_\pi(s')$
- so we can estimate advantage from the value function alone,

$$A_\pi(s_{i,t}, a_{i,t}) \approx r_{i,t} + \gamma V_\pi(s_{i,t+1}) - V_\pi(s_{i,t})$$

Actor-Critic Models

$$\nabla_\theta L(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \big(r_{i,t} + \gamma V_\pi(s_{i,t+1}) - V_\pi(s_{i,t}) \big)$$

- train one network to estimate π_θ and another to estimate V_π

Actor-Critic Models

$$\nabla_\theta L(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t}) \big(r_{i,t} + \gamma V_\pi(s_{i,t+1}) - V_\pi(s_{i,t})\big)$$

- train one network to estimate π_θ and another to estimate V_π
- collect trajectories into a dataset, $\{(s_{i,t}, y_{i,t})\}$ where $y_{i,t} = \sum_{\tau=t}^{T} \gamma^{\tau-t} r_{i,\tau}$
- use supervised learning for network with parameters ϕ to estimate V_π:
  - (i) basic Monte Carlo estimation

$$L(\phi) = \frac{1}{2} \sum_{i=1}^{N} \sum_{t=1}^{T} \|\hat{V}_\pi(s_{i,t}; \phi) - y_{i,t}\|^2$$

  - (ii) bootstrapping

$$L(\phi) = \frac{1}{2} \sum_{i=1}^{N} \sum_{t=1}^{T} \|\hat{V}_\pi(s_{i,t}; \phi) - (r_{i,t} + \gamma \hat{V}_\pi(s_{i,t+1}; \phi^{\text{prev}})\|^2$$

- alternate between updating θ and updating ϕ

Proximal Policy Optimization (PPO)

(Schulman et al., 2017)

- state-of-the-art for deep reinforcement learning
- stabilizes training by limiting how much a policy can change during each update
- define **probability ratio** $\rho_t(\theta)$ relative to some previous model θ_{prev} as

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{prev}}}(a_t \mid s_t)}$$

- PPO implements the clipped loss

$$L(\theta) = E\left[\min\{\rho_t(\theta)A_\pi(s_t, a_t),\ \text{clip}_{[1-\epsilon,1+\epsilon]}(\rho_t(\theta))A_\pi(s_t, a_t)\}\right]$$

- other loss variants are possible (e.g., using a KL divergence penalty)
- variant called GRPO (Shao et al., 2024) is used at last stage of LLM finetuning

Further Reading

Deep reinforcement learning is a very active area of research. For those interested, some advanced topics can be found in the following papers:

- Haarnoja et al., "Soft Actor-Critic Algorithms and Applications," 2019
- Lillicrap et al., "Continuous control with deep reinforcement learning," 2015
- Hafner et al., "Dream to Control: Learning Behaviors by Latent Imagination," 2020
- Shao et al., "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," 2024